



**INSTITUTO
FEDERAL**
Santa Catarina

Governança de Repositórios

Engenharia de Software II

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br



cores

tom escuro	#384254	oxford blue
acento escuro	#9599AE	santas gray
cor principal	#E74146	cinnabar
acento claro	#E2AC5B	equator
tom claro	#F8FAF9	snow drift
	#252A34	

fontes

Fira Sans Extra Condensed

Ubuntu

Roboto Mono

Governança de Repositórios

cenário

É sexta-feira à noite. Segunda tem demonstração. Alguém só fez um "ajuste rápido" e subiu direto na *main*. Agora a *main* não roda. Ninguém sabe exatamente o que mudou, por quê, e qual PR/issue explica. O time inteiro para pra caçar o problema.

que tipo de problema é esse?

a) problema de ferramenta

o git deixou de fazer *push* na *main*, então o problema está no git

b) problema de governança

não há proteção na *main*, nem obrigação de fazer *pull request*, revisão de código ou checagens

c) problema de comunicação

se todo mundo avisasse antes de mexer, não haveria problema de quebrar a *main*

d) problema de calendário

demos na segunda são contraproduativas; a solução é alterar a demo para quarta e oferecer café aos participantes

Governança de Repositório

o que é

- conjunto de regras, papéis, padrões e automações aplicado ao repositório
- definem
 - como um time trabalha dentro de um repositório
 - como mudar o código com segurança, rastreabilidade e qualidade mínima

importância

- evita quebrar a ramificação principal (*main*, *master*, etc)
- cria rastreabilidade (auditoria)
- reduz retrabalho e conflitos
- melhora qualidade com custo baixo
- permite escalar o time

Governança de Repositório

governança mínima

■ papéis e autoridade

- quem pode dar merge na *main*
- quem pode aprovar PR
- o que fazer quando há impasse

sugestões

- ninguém pode dar *merge* diretamente na *main* (somente via *pull request*)
- qualquer um pode abrir PR
- *merge* só depois de uma aprovação (autor não aprova o próprio PR)
- se houver impasse, o "líder" resolve
- para código sensível, pode-se definir os "donos do código" (CODEOWNERS) no github

Governança de Repositório

sobre o arquivo CODEOWNERS

- deve estar na raiz do projeto ou nos diretórios / .github ou /docs
- várias regras podem estar no arquivos
 - a última regra tem precedência
- uma regra consiste em um padrão de arquivo seguido dos seus proprietários
- podem ser usados nomes de usuário, nomes de equipes e e-mails (com acesso de escrita) para definir os proprietários
- padrões de arquivo são como em .gitignore

exemplo

```
# 1) Dono padrão (se nenhuma regra específica casar)
*                                @tech-leads

# 2) Documentação
/docs/                           @time-docs
README.md                       @time-docs

# 3) Frontend
/frontend/                       @time-frontend

# 4) Backend
/backend/                       @time-backend

# 5) Infra/Config
.github/                         @tech-leads
/compose.yml                    @time-infra
/.env.example                   @time-infra

# 6) Banco de dados
/database/                      @time-backend @time-infra

# 7) Testes
/tests/                         @time-qa
```

Governança de Repositório

governança mínima

- papéis e autoridade
- regras de ramificações
 - uma *branch* por tarefa/*issue*
 - nomes padronizados
 - fix • feat • test
 - build • chore • refactor
 - ci • docs • styles
 - mantenha a branch curta

sugestões de fluxo

- 1. criar branch
 - feat/23-cadastro-patrimonio
- 2. faça commits pequenos
 - adiciona formulário de cadastro
 - valida campos obrigatórios
 - salva patrimônio no storage
- 3. abra PR com descrição
 - o que mudou e por quê; como testar
- 4. revisão e ajustes
- 5. aprovar o PR e apagar a *branch*

Governança de Repositório

governança mínima

- papéis e autoridade
- regras de ramificações
- **regras de *commits***
 - faça *commits* pequenos e descritivos
 - não junte coisas diferentes no mesmo *commit*
 - mantenha a branch curta
 - quando algo quebra, o histórico ajuda a investigar

sugestões de fluxo

- 1. criar branch
 - feat/23-cadastro-patrimonio
- 2. faça commits pequenos
 - adiciona formulário de cadastro
 - valida campos obrigatórios
 - salva patrimônio no storage
- 3. abra PR com descrição
 - o que mudou e por quê; como testar
- 4. revisão e ajustes
- 5. aprovar o PR e apagar a *branch*

Governança de Repositório

governança mínima

- papéis e autoridade
- regras de ramificações
- regras de *commits*
- **definição de pronto (DoD)**
 - critérios para entrar na *main*
 - critérios mínimos de clareza, rastreabilidade e qualidade
 - evita a maior parte dos problemas clássicos
 - PR “misterioso”
 - bug bobo entrando
 - discussão infinita

critérios

- PR com descrição (clareza)
 - o que mudou
 - ajusta regra de depreciação para bens com mais de 5 anos e aplica valor residual
 - por quê
 - cálculo estava retornando valores negativos em alguns casos (#31)
 - impacto
 - afeta endpoint
/patrimonios/{id}/depreciacao
 - por quê
 - passos para o teste manual e/ou automatizado

Governança de Repositório

governança mínima

- papéis e autoridade
- regras de ramificações
- regras de *commits*
- **definição de pronto (DoD)**
 - critérios para entrar na *main*
 - critérios mínimos de clareza, rastreabilidade e qualidade
 - evita a maior parte dos problemas clássicos
 - PR “misterioso”
 - bug bobo entrando
 - discussão infinita

critérios

- PR com descrição (clareza)
- PR vinculado a uma *issue* (rastreabilidade)
 - incluir no PR
 - Fixes #23 (fecha a issue ao merge)
 - Related to #23 (só relaciona)
 - justifica o PR
 - ajuda a montar relatório de sprint

Governança de Repositório

governança mínima

- papéis e autoridade
- regras de ramificações
- regras de *commits*
- **definição de pronto (DoD)**
 - critérios para entrar na *main*
 - critérios mínimos de clareza, rastreabilidade e qualidade
 - evita a maior parte dos problemas clássicos
 - PR “misterioso”
 - bug bobo entrando
 - discussão infinita

critérios

- PR com descrição (clareza)
- PR vinculado a uma *issue* (rastreabilidade)
- checks/CI passaram
 - lint/format (pega erro bobo de sintaxe/estilo)
 - testes unitários (evita regressão)
 - build (garante que compila/gera bundle)
 - testes de integração (quando tiver)

Governança de Repositório

governança mínima

- papéis e autoridade
- regras de ramificações
- regras de *commits*
- **definição de pronto (DoD)**
 - critérios para entrar na *main*
 - critérios mínimos de clareza, rastreabilidade e qualidade
 - evita a maior parte dos problemas clássicos
 - PR “misterioso”
 - bug bobo entrando
 - discussão infinita

critérios

- PR com descrição (clareza)
- PR vinculado a uma *issue* (rastreabilidade)
- checks/CI passaram
- PR pequeno o suficiente
 - grande demais se não dá pra revisar em ~10 min
 - demora pra revisar
 - aumenta chance de bug passar
 - vira discussão interminável
 - dificulta reverter

Governança de Repositório

governança mínima

- papéis e autoridade
- regras de ramificações
- regras de *commits*
- definição de pronto (DoD)
- **regras de *review***
 - processo técnico de redução de risco
 - deve responder duas perguntas
 - isso pode quebrar algo? (risco)
 - isso vai ser fácil de manter no futuro? (manutenibilidade)

regras

- 1. comente risco, não estilo pessoal
- 2. sempre explique o motivo e, se possível, proponha alternativa
- 3. se a refatoração não for necessária, deixe para outro PR
- 4. verifique se o PR está pequeno e com escopo claro
- 5. cobre evidência de teste

Governança de Repositório

proteção de ramificações

- opções do github
 - *branch protection rule*
 - *branch ruleset*



**INSTITUTO
FEDERAL**
Santa Catarina

Governança de Repositórios

Engenharia de Software II

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br

